

Comparative Study of Multivariable Linear Regression Implementations

Shreya Johari

May 2026

1 Introduction

This project compares three different implementations of multivariable linear regression:

- Pure Python implementation
- NumPy vectorized implementation
- Scikit-learn implementation

The objective of this project is to analyze convergence speed, execution time, and predictive performance across the three approaches.

2 Dataset Description

The Melbourne Housing Dataset was used for training and evaluation.

2.1 Features Used

- Rooms
- Distance
- Bathroom
- Car

2.2 Target Variable

- Price

Rows containing missing values were removed before training.

3 Data Preprocessing

Feature scaling was performed using standardization. Each feature was normalized using mean and standard deviation.

Scaling improves gradient descent convergence and numerical stability.

4 Methodology

4.1 Pure Python Implementation

The Pure Python implementation was developed using only core Python features. Gradient descent optimization was implemented manually using nested loops. Weights were updated iteratively to minimize Mean Squared Error.

4.2 NumPy Vectorized Implementation

The NumPy implementation used vectorized matrix operations to eliminate explicit Python loops. This significantly improved computational efficiency and convergence speed.

4.3 Scikit-learn Implementation

The Scikit-learn implementation used the `LinearRegression` class. This implementation internally uses optimized linear algebra techniques for fitting the regression model efficiently.

5 Evaluation Metrics

The following metrics were used for evaluation:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R-squared (R^2) Score

5.1 MAE Formula

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

5.2 RMSE Formula

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

5.3 R² Formula

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

6 Results

6.1 Performance Comparison Table

Method	MAE	RMSE	R ²	Time (s)
Pure Python	XXX	XXX	XXX	XXX
NumPy	XXX	XXX	XXX	XXX
Scikit-learn	XXX	XXX	XXX	XXX

Table 1: Performance Comparison of Implementations

7 Analysis

The Pure Python implementation was significantly slower due to explicit looping operations.

The NumPy implementation achieved major speed improvements because vectorized matrix operations are optimized in low-level compiled code.

Scikit-learn achieved the fastest fitting time and best numerical stability due to highly optimized backend implementations.

Feature scaling improved gradient descent convergence and stability.

8 Conclusion

This project demonstrated the impact of optimization techniques on machine learning implementations.

NumPy vectorization significantly improved performance compared to Pure Python, while Scikit-learn provided the most efficient and reliable implementation.

9 AI Usage Disclosure

ChatGPT was used for debugging assistance, code structuring suggestions, and conceptual clarification during development.

All implementations and analysis were manually verified and understood.